



LAYOUT COM FLEXBOX

O FIM DO SOFRIMENTO PARA ALINHAR ELEMENTOS

Tiago Henrique Angioletti





O QUE VAMOS APRENDER HOJE?

Os problemas de layout antes do Flexbox.

A **REGRA DE OURO**: Elemento Pai vs. Elementos Filhos.

Como colocar itens lado a lado ou em coluna (`flex-direction`).

Como alinhar elementos horizontalmente (`justify-content`).

Como alinhar elementos verticalmente (`align-items`).

O atalho mágico para espaçamento (`gap`).



O MUNDO ANTES DO FLEXBOX

Antigamente, colocar duas caixas lado a lado no CSS era uma dor de cabeça. Usávamos gambiarras com propriedades como `float`, cálculos matemáticos de porcentagem e margens negativas.

O **FLEXBOX** (Flexible Box Module) chegou para resolver isso. Ele foi criado especificamente para criar layouts unidimensionais (uma linha ou uma coluna) de forma inteligente e fluida.



A REGRA DE OURO DO FLEXBOX

O conceito mais importante para não se perder: **VOCÊ SEMPRE APLICA O COMANDO NO ELEMENTO PAI (O CONTAINER) PARA DITAR AS REGRAS DOS ELEMENTOS FILHOS.**

```
<nav class="menu">
  <a href="#">Home</a>
  <a href="#">Sobre</a>
  <a href="#">Contato</a>
</nav>
```

Se eu quero que os links fiquem lado a lado, eu aplico o Flexbox na classe `.menu` !



ATIVANDO O PODER: DISPLAY E DIRECTION

```
.menu {  
  /* 1. Ativa o Flexbox no elemento pai */  
  display: flex;  
  
  /* 2. Define a direção em que os filhos vão fluir */  
  flex-direction: row; /* Padrão: coloca os itens em LINHA (lado a lado) */  
  
  /* Outra opção: */  
  /* flex-direction: column; -> Coloca os itens empilhados (um embaixo do outro) */  
}
```



JUSTIFY-CONTENT (EIXO PRINCIPAL)

Se o seu `flex-direction` é `row` (linha), o `justify-content` alinha os itens na **HORIZONTAL**.

```
.menu {  
  display: flex;  
  flex-direction: row;  
  
  /* Como distribuir o espaço horizontal? */  
  justify-content: flex-start; /* Padrão: tudo grudado na esquerda */  
  /* justify-content: center; -> Tudo centralizado */  
  /* justify-content: flex-end; -> Tudo grudado na direita */  
  /* justify-content: space-between; -> O 1º na esquerda, o último na direita, e espaço  
  /* justify-content: space-around; -> Espaço igual ao redor de todos os itens */  
}
```



ALIGN-ITEMS (EIXO SECUNDÁRIO)

Se o seu `flex-direction` é `row`, o `align-items` alinha os itens na **VERTICAL** (para cima ou para baixo dentro da caixa pai).

```
.container-alto {  
  display: flex;  
  height: 200px; /* Uma caixa bem alta */  
  
  /* Como alinhar verticalmente? */  
  align-items: stretch; /* Padrão: Os filhos esticam para preencher a altura */  
  /* align-items: flex-start; -> Ficam no topo */  
  /* align-items: flex-end; -> Ficam no fundo */  
  /* align-items: center; -> Ficam perfeitamente centralizados no meio da caixa! */  
}
```



O TOQUE MÁGICO: GAP

Antes do Flexbox, para dar espaço entre os botões de um menu, tínhamos que colocar `margin-right` em todos e tirar do último. Com o Flexbox, usamos o `gap`.

```
.menu {  
  display: flex;  
  justify-content: center;  
  
  /* Cria um "buraco" de 20px exatos ENTRE os elementos filhos, mas não nas pontas! */  
  gap: 20px;  
}
```




MÃO NA MASSA: PASSO A PASSO (1/2)

Vamos criar uma barra de navegação (Header) profissional usando Flexbox.
No seu arquivo HTML, adicione este código:

```
<header class="cabecalho">
  <h1 class="logo">MeuSite</h1>
  <nav class="menu-navegacao">
    <a href="#">Início</a>
    <a href="#">Projetos</a>
    <a href="#">Contato</a>
  </nav>
</header>
```



MÃO NA MASSA: PASSO A PASSO (2/2)

Agora, no seu `style.css`, faça a mágica acontecer:

```
/* 1. Reset básico para os links */
a { text-decoration: none; color: #333; font-weight: bold; }

/* 2. O Pai de Todos (Header) */
.cabecalho {
  display: flex;
  justify-content: space-between; /* Joga o Logo pra esquerda e Menu pra direita */
  align-items: center; /* Centraliza eles verticalmente */
  padding: 20px 40px;
  background-color: #f8f9fa;
}

/* 3. O Pai do Menu (Nav) */
.menu-navegacao {
  display: flex;
```



? REVISÃO RÁPIDA

Vamos ver se fixamos o Box Model e o Flexbox:

Em qual elemento eu aplico o `display: flex;` ? No contêiner pai ou nos itens filhos?

Se eu quero centralizar uma imagem no meio da tela perfeitamente, quais propriedades de alinhamento eu uso?

Qual a diferença prática entre usar `margin` nos filhos ou `gap` no pai?